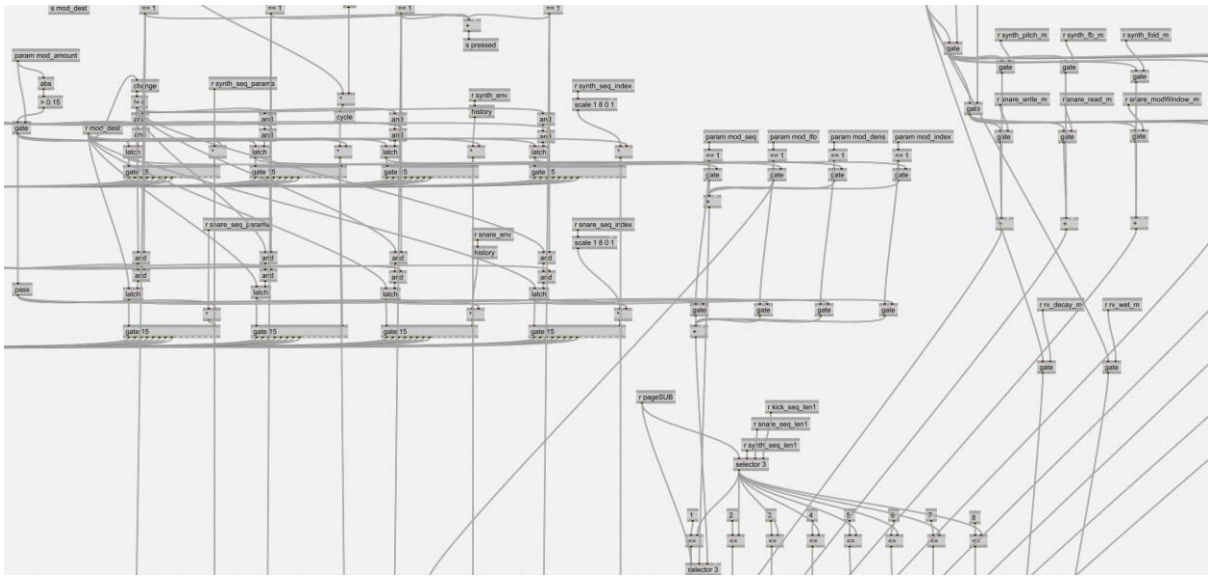
CMPN4608 Composition Portfolio

Michael Smith – “Orrery” Hardware Synthesizer

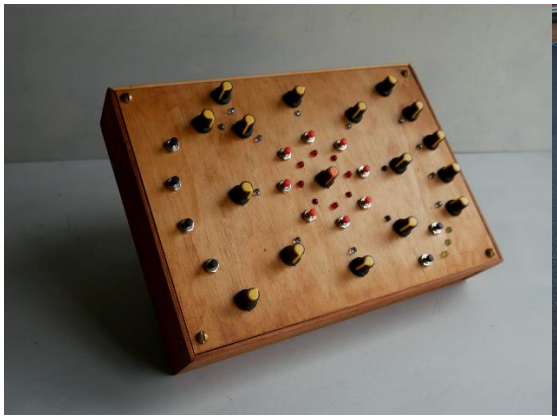
**Overview**

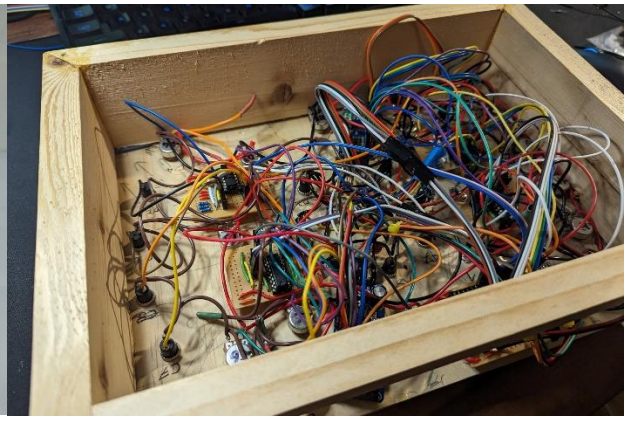
*Orrery* is a hardware synthesizer built using the Daisy platform (https://electro-smith.com/), and coded in Max/MSP gen.



The synthesizer was programmed on my PC in gen (above is an example of part of the patch) and then loaded onto the Daisy Seed, which is a small computer sold by Electrosmith designed for audio processing.

The synth itself then has a physical wooden body and a range of components (knobs, buttons, LEDs) that are then soldered into the pins of the Seed, connecting them into the code.



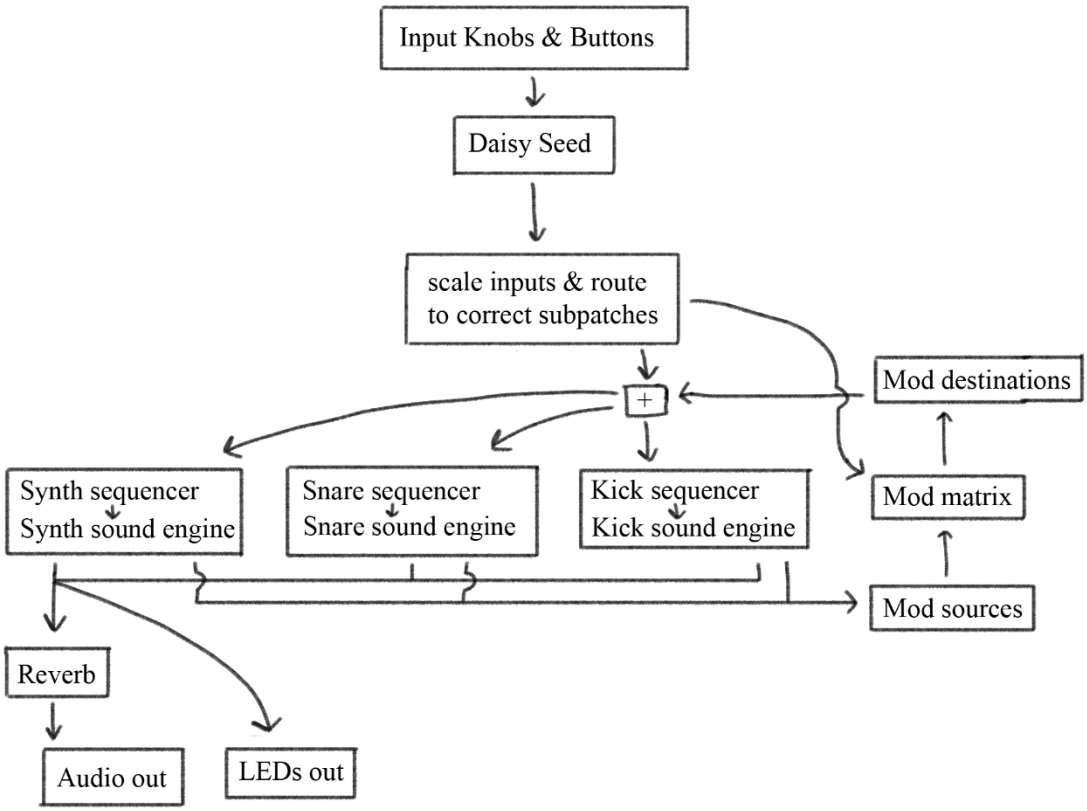


Functionally, the machine contains three instruments: a synth voice, a snare voice and a kick voice. Each of these instruments has an associated sequencer and set of controls, allowing the user to create a pattern combining the three. This makes the machine more of a “groovebox” than a typical synthesizer.

A set of controls also allows the user to manipulate the underlying clock that controls the sequencers. There are two points placed in the sequence (one at the start and one halfway through) that can be made more “massive”, attracting nearby clock pulses towards them. This results in bunched up or spaced-out clock pulses.

There is also a modulation matrix, where a range of sources can be mapped to modulate a range of destinations. For example, a slowly shifting LFO can be made to control the synth's pitch. As some of the modulation sources are related to the sequence, and some of the modulation destinations can impact the sequence, it is possible to set up a modulation system that exhibits highly feedbacked and chaotic behaviour.

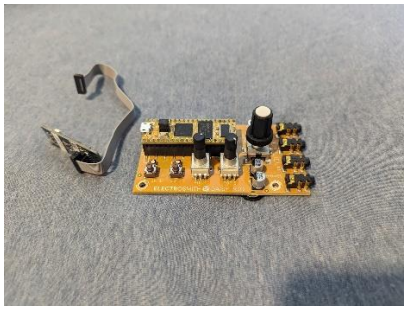
Below is a top-level diagram that shows the architecture of the gen patch:




**Process**

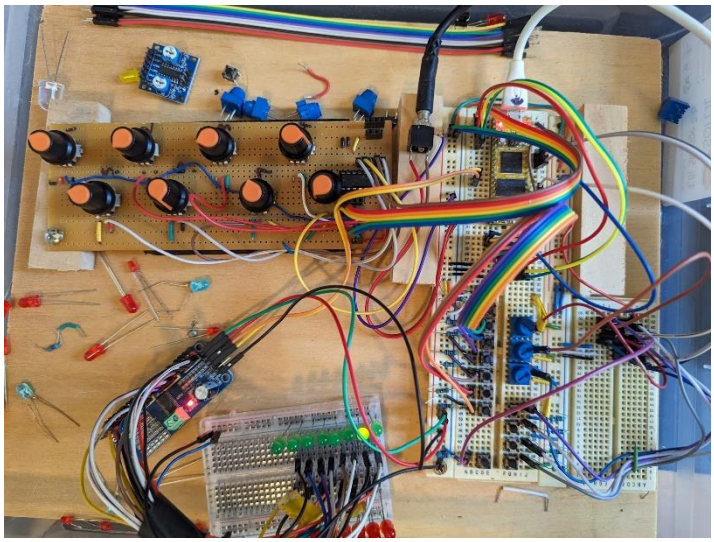
An overview of the process of creating the synth is as follows:

- Obtained the Daisy Seed



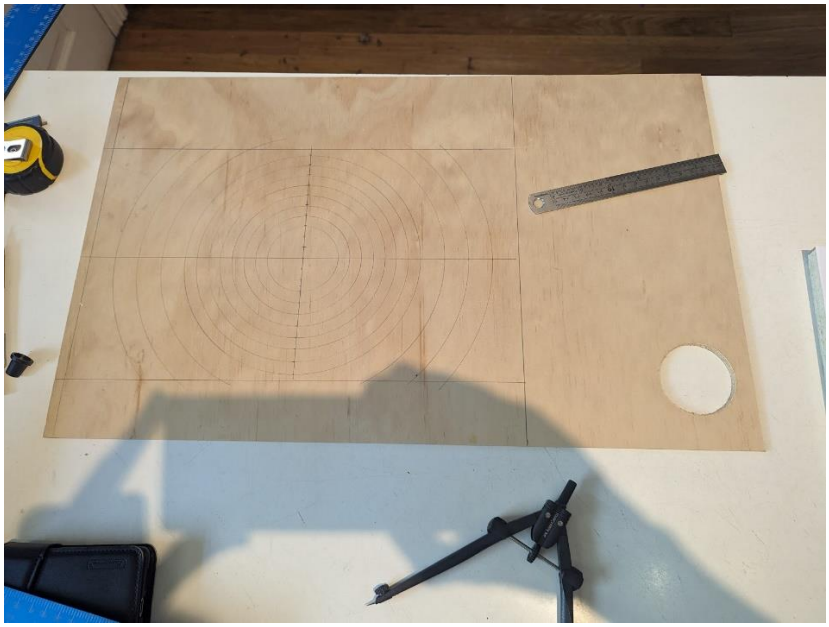
- Learned how to program it with basic patches (1 knob in, audio out, etc.) – The seed initially came with a breakout board called the pod that provided audio out and a few components which helped to test in the early stages.

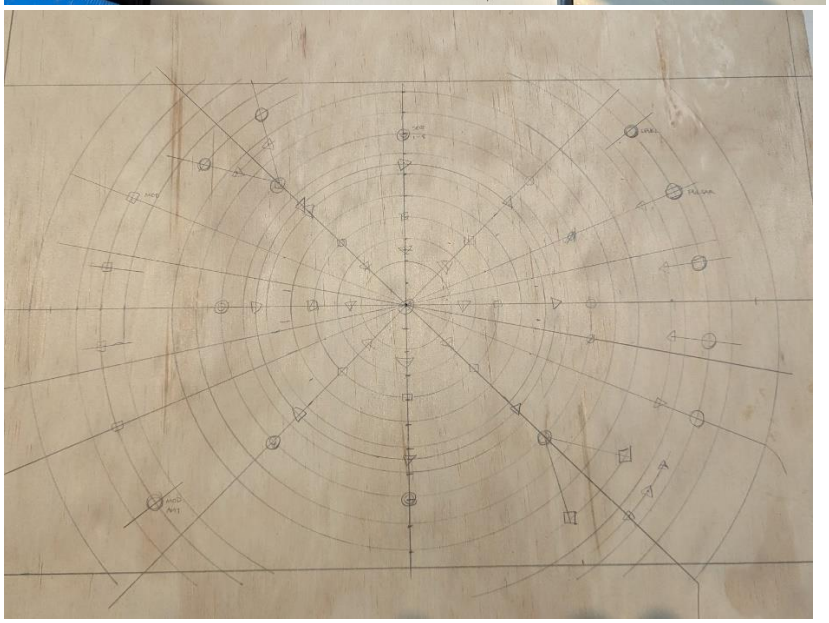
- Built the gen patch: this took the majority of the semester's time. I went through numerous iterations, adding and removing functionality each time. At this point it was purely built in software for ease of testing.
- Realised the number of inputs and outputs supported on the Seed would be nowhere near enough to meet the requirements of my program. I investigated getting some multiplexers, chips that allow 8 components to fit on a single pin on the Seed, allowing a much higher upper limit of components.
- Slowly built up a breadboard equipped with a growing number of knobs and buttons to test pieces of the code.
- Ordered components to be delivered. Didn't know exactly how many at this point but was able to ballpark it.
- Attemped to run the full patch on the seed to no avail. This would be the first of many rounds of optimization as I refactored and restructured the program to be able to work on the seed.
- Fleshed out my breadboard mockup with enough components to control everything.



- Developed the program into a near-final state, then totally disassembled the mockup in order to move into the permanent casing.

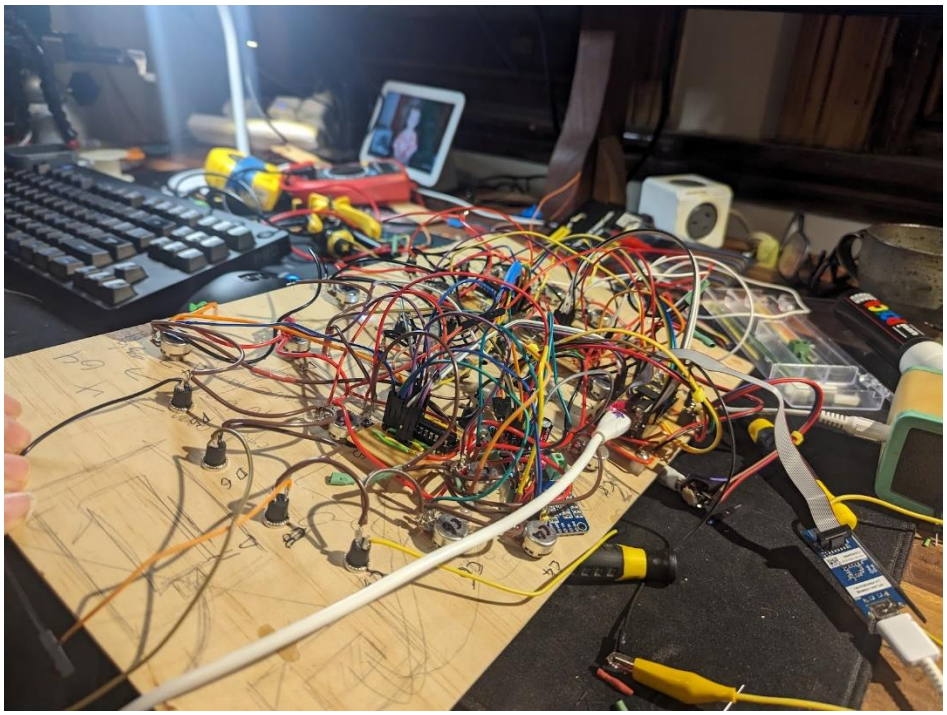
- Designed the front panel:





- Cut holes in a plywood panel

- Spent many (many!) hours soldering every component in place (pictured was about halfway)



- Once all components were in place, I attempted to run the program on the Seed to no success. Investigation revealed there was a CPU usage limit that I was exceeding, and numerous sucessive rounds of optimization were required to continually put me back below that limit after adding each new feature.

- Once semi-stable, built a temporary wooden box to hold the panel

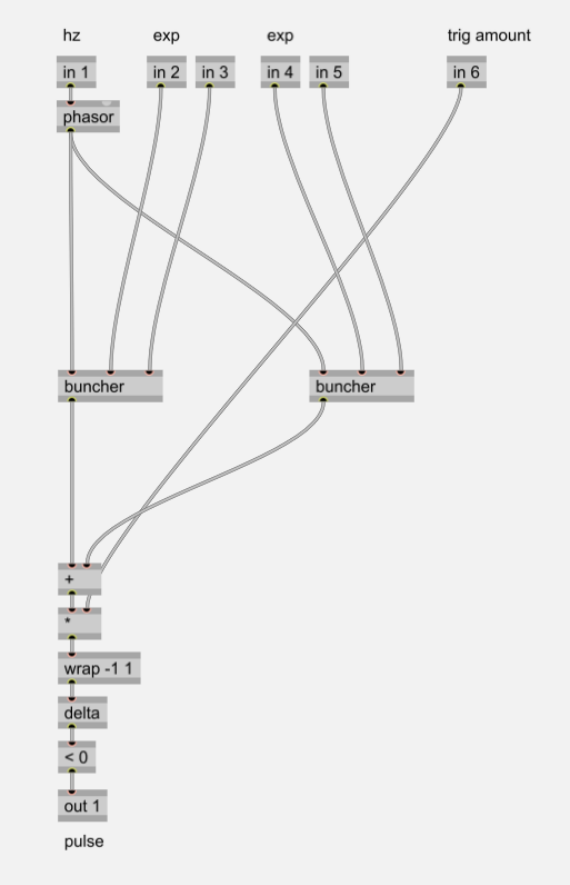
- Now in the final week or two – did a lot of further optimization and refinement of features. Only in the final week is the instrument truly functional and playable.
- Finally – redid the box into a more nice-looking permanent casing.

Note: I was lucky enough to receive quite a bit of help on this project from my father who is experienced in both software development and woodworking. To transparently lay out his contributions, he constructed the wooden box with my help and then taught me how to solder to assemble the rest of the synth. He also helped out with a lot of research and forum reading to gather resources on how to understand the Daisy Seed platform. The rest of the project, including all of the coding, assembly, and design were all done by me.

**Details of functionality**

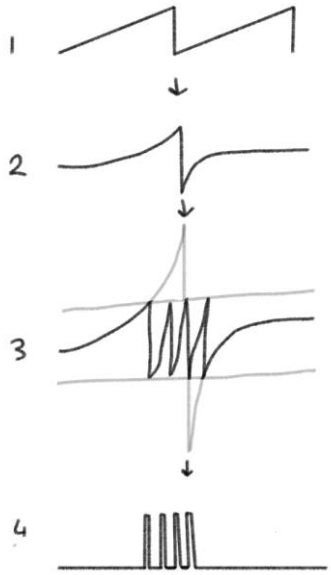
Here is a breakdown of what each section of the program does:

**Pulsar**



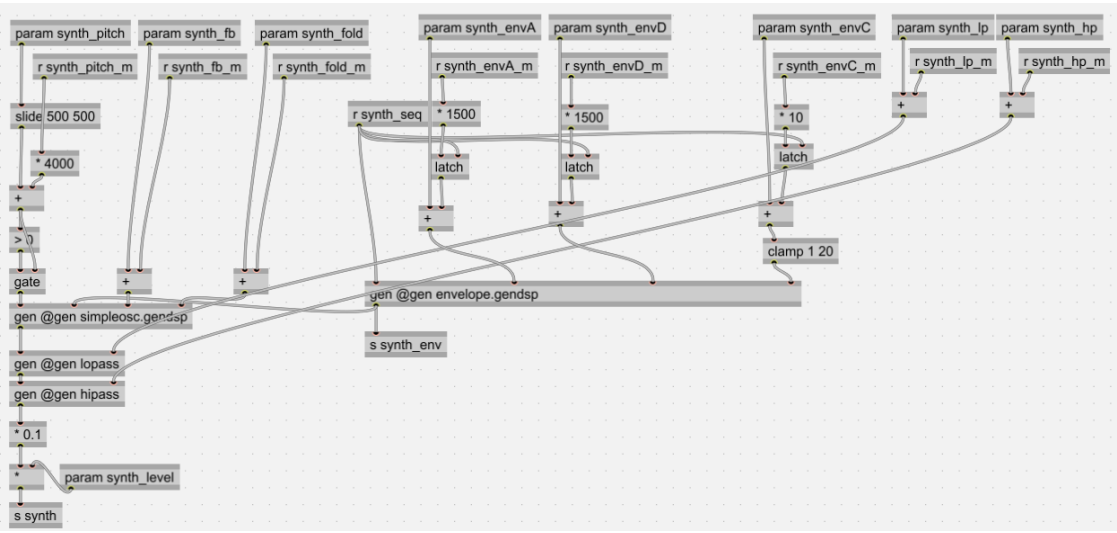

Takes four arguments: Clock rate, strengths of the two attractors and the number of pulses to generate per attractor.
Outputs a series of single-sample pulses.

Illustration of the 'buncher' algorithm:



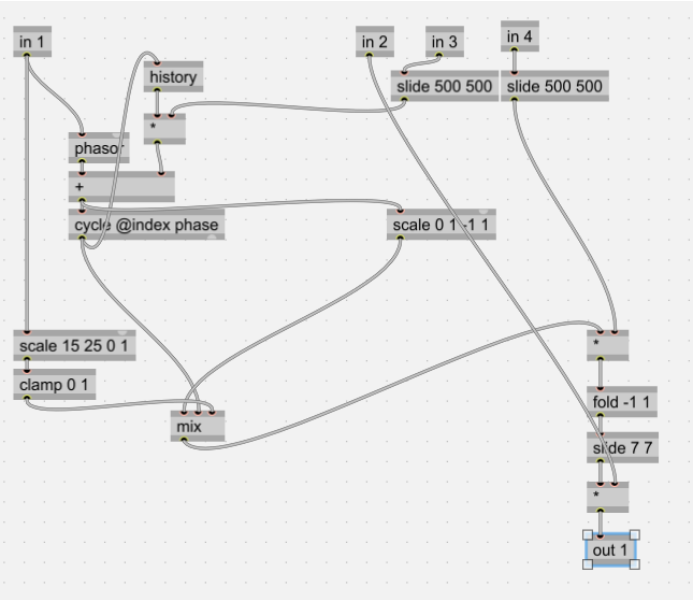

A saw wave of desired frequency is first exponentiated to achieve the appropriate level of curvature, then this wave is multiplied beyond the bounds of -1 to 1 and then wrapped back into that range. An edge detection algorithm is then applied to create the pulses.

**Synth**

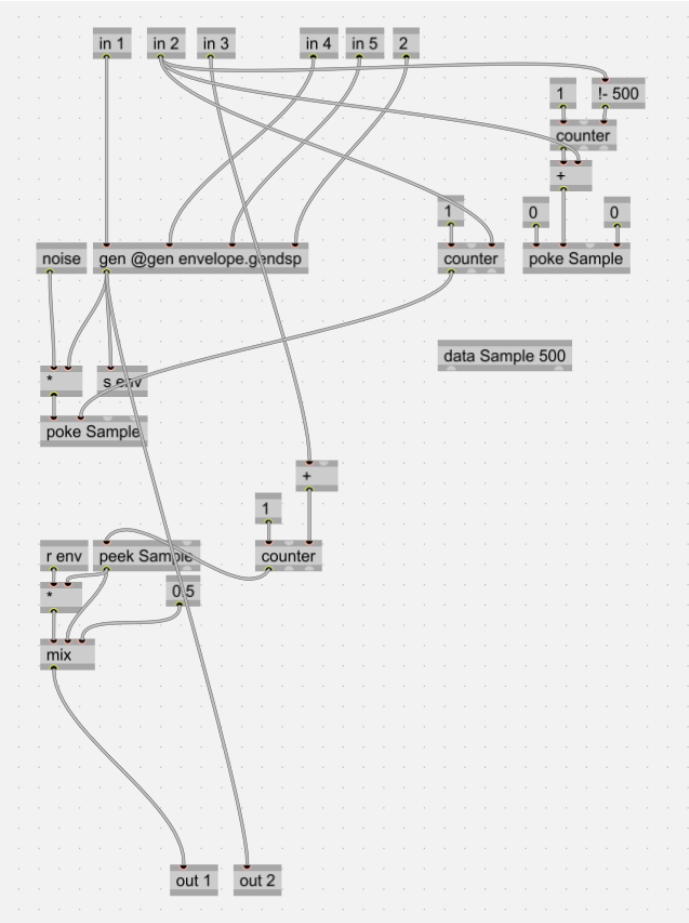


The synth description will also cover the general architecture used for all of the instruments. The relevant parameters are imported, and then sent into their respective destinations after being summed with an offset that is initially zero but will change should those values be modulated.

The synth voice is being controlled by an envelope in a separate abstraction that I designed and then sent through a lowpass and highpass filter.



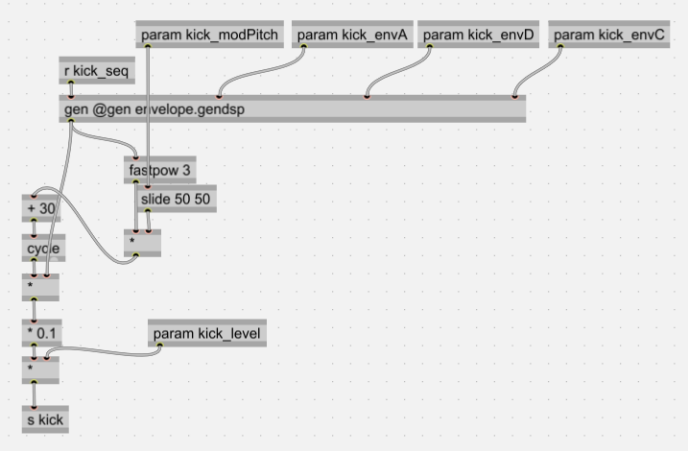

Inside the oscillator subpatch. Takes arguments of pitch (hz), feedback amount, and wavefolding amount. The only unusual thing of note here is that there is a filter that swaps the sine wave out for a saw wave below 15 hz for more definition.




**Snare**

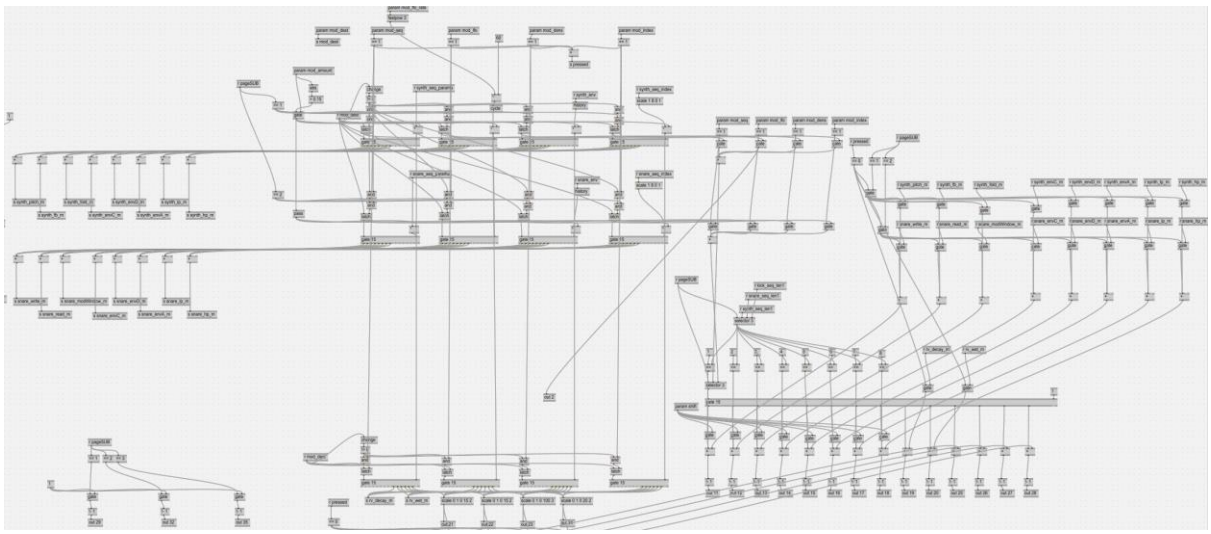
Inside the snare subpatch. I create a 500 sample long buffer. Then, there are two looping counters running simultaneously – One causes noise to be continuously written into the buffer and the other reads out of the buffer. By setting these counter lengths, interesting phasing artifacts are created as the noise is read and written.

## Kick




By far the simplest of the instruments, the kick is merely a sine wave whose frequency is tied to the envelope to create the classic kick falloff sound. I would have loved to flesh out the kick with more parameters but sadly the CPU limit of the seed has meant I have had to cut this idea.

## Mod Matrix



To give a rough overview of the mod matrix: A series of gates route up to 8 modulation sources to 14 modulation destinations (8 in each of the synth and snare + 6 global destinations). The system is essentially made of many layered latches and logic gates that ensure the values are being sent to the right places based on which buttons are being held down. The block on the right then manages all the LEDs.

## Other Materials:

As well as this document, I have created a series of videos demonstrating the Orrery – a 10-minute tutorial style video with voiceover and a further series of music-only performances. I am also submitting the max file for the project, which contains a software recreation of all the physical components, allowing you to emulate using the real machine.